LAPORAN PRAKTIKUM

PRAKTIKUM PENAMBANGAN DATA

PERTEMUAN 3

CLASSIFICATION



Disusun oleh:

Nama	:	Hanan Fijananto
NIM	:	24/538946/SV/24555
Kelas	:	A2
Dosen Pengampu	:	Dr.Imam Fahrurrozi, S.T., M.Cs.

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK

DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA

SEKOLAH VOKASI

UNIVERSITAS GADJAH MADA

YOGYAKARTA

2026

A Tujuan

1. Mahasiswa mampu memahami konsep dasar beberapa algoritma klasifikasi, seperti Logistic Regression, KNN, Decision Tree, dan Random Forest
2. Mahasiswa mampu melakukan evaluasi model dengan menyusun tabel performa berisi accuracy, precision, dan recall dari setiap model
3. Mahasiswa mengetahui fungsi dan peran metrik evaluasi klasifikasi, seperti accuracy, precision, dan recall dalam menilai performa model
4. Mahasiswa mampu melakukan evaluasi model klasifikasi menggunakan metrik yang sesuai

B Dasar Teori

B.1 Classification

Klasifikasi adalah salah satu metode dalam data mining yang digunakan untuk mengelompokkan data ke dalam kelas atau kategori tertentu berdasarkan pola yang telah dipelajari sebelumnya.

Metode ini bekerja dengan memanfaatkan data historis yang sudah memiliki label atau kategori. Data tersebut digunakan untuk melatih sebuah model sehingga model tersebut dapat mengenali pola tertentu. Setelah model terbentuk, sistem dapat menggunakannya untuk memprediksi kategori dari data baru yang belum pernah dilihat sebelumnya [1].

B.2 Algoritma Classification

Klasifikasi (*classification*) merupakan salah satu teknik utama dalam pembelajaran mesin terarah (*supervised learning*) dan penggalian data (*data mining*).

Tujuan utama dari algoritma klasifikasi adalah untuk memetakan suatu data masukan ke dalam target kategori atau kelas yang telah ditentukan sebelumnya. Proses ini didasarkan pada pembelajaran model menggunakan sekumpulan data pelatihan (*training data*) yang sudah memiliki label kebenaran aktual [2].

Secara umum, proses klasifikasi terdiri dari dua tahapan utama. Tahap pertama adalah fase pembelajaran (*learning phase*) di mana algoritma membangun model klasifikasi berdasarkan data pelatihan. Tahap kedua adalah fase pengujian (*testing phase*) di mana model yang telah dibangun dievaluasi menggunakan data uji (*testing data*) untuk memprediksi label kelas yang belum diketahui, sehingga tingkat akurasi model tersebut dapat diukur. Beberapa algoritma klasifikasi yang umum digunakan dalam penelitian antara lain *K-Nearest Neighbors* (KNN), *Naive Bayes*, *Decision Tree*, dan *Support Vector Machine* (SVM).

B.3 Matriks Evaluasi Klasifikasi

Untuk mengukur seberapa baik kinerja dari sebuah algoritma klasifikasi, diperlukan alat ukur atau matriks evaluasi. Metode evaluasi dasar yang paling sering digunakan untuk masalah klasifikasi adalah *Confusion Matrix* (matriks kebingungan). Matriks ini menyajikan perbandingan antara nilai prediksi yang dihasilkan oleh model dengan nilai kelas aktual dari data uji [3].

Dalam klasifikasi biner, *Confusion Matrix* menghasilkan empat komponen utama, yaitu:

- ***True Positive* (TP):** Jumlah data positif yang diprediksi benar sebagai positif oleh model.
- ***True Negative* (TN):** Jumlah data negatif yang diprediksi benar sebagai negatif oleh model.
- ***False Positive* (FP):** Jumlah data negatif yang secara keliru diprediksi sebagai positif oleh model (Galat Tipe I).

- ***False Negative* (FN):** Jumlah data positif yang secara keliru diprediksi sebagai negatif oleh model (Galat Tipe II).

Berdasarkan keempat komponen matriks tersebut, beberapa metrik evaluasi kinerja dapat dihitung menggunakan persamaan matematis sebagai berikut:

1. **Akurasi (*Accuracy*)**

   Akurasi mengukur rasio prediksi yang benar secara keseluruhan (baik kelas positif maupun negatif) terhadap total keseluruhan data yang diuji.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{1}$$

2. **Presisi (*Precision*)**

   Presisi mengukur tingkat ketepatan prediksi kelas positif. Metrik ini menunjukkan persentase data yang benar-benar positif dari seluruh data yang diprediksi positif oleh model.

$$Precision = \frac{TP}{TP + FP} \tag{2}$$

3. ***Recall* (Sensitivitas)**

   Recall mengukur kemampuan model dalam mengidentifikasi atau menemukan kembali seluruh data aktual yang bernilai positif.

$$Recall = \frac{TP}{TP + FN} \tag{3}$$

4. ***F1-Score***

   F1-Score merupakan rata-rata harmonik dari presisi dan *recall*. Metrik ini sangat berguna dan menjadi acuan utama ketika set data yang digunakan

memiliki distribusi kelas yang tidak seimbang (*imbalanced dataset*).

$$F1\text{-}Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{4}$$

C Hasil dan Pembahasan

C.1 Praktikum

C.1.1 Import Library

Sebelum mengimport library, install library yang dibutuhkan seperti berikut.

```
!pip install imbalanced-learn
```

Library yang digunakan dalam praktikum ini adalah sebagai berikut:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import confusion_matrix
from sklearn.metrics import ConfusionMatrixDisplay
from imblearn.metrics import sensitivity_specificity_support
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn. tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import AdaBoostClassifier
```

Gambar 1: Import Library

C.1.2 Load Dataset

Dataset yang digunakan dalam praktikum ini yaitu diambil dari kaggle dengan link `https://www.kaggle.com/datasets/fedesoriano/stroke-prediction-dataset`.

Dataset tersebut kemudian dibaca dengan pandas

```
df = pd.read_csv
    ('https://raw.githubusercontent.com/ganjar87/
data_science_practice/refs/heads/main/
healthcare-dataset-stroke-data.csv')
```

C.1.3 Explatory Data Analysis (EDA)

1. **Melihat 5 data teratas dan terbawah**

```
df.head()
```
✓ 0.0s

	id	gender	age	hypertension	heart_disease	ever_married	work_type	Residence_type	avg_g
0	9046	Male	67.0	0	1	Yes	Private	Urban	
1	51676	Female	61.0	0	0	Yes	Self-employed	Rural	
2	31112	Male	80.0	0	1	Yes	Private	Rural	
3	60182	Female	49.0	0	0	Yes	Private	Urban	
4	1665	Female	79.0	1	0	Yes	Self-employed	Rural	

```
df.tail()
```
✓ 0.0s

	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	Coapplica
376	LP002953	Male	Yes	3+	Graduate	No	5703	
377	LP002974	Male	Yes	0	Graduate	No	3232	
378	LP002978	Female	No	0	Graduate	No	2900	
379	LP002979	Male	Yes	3+	Graduate	No	4106	
380	LP002990	Female	No	0	Graduate	Yes	4583	

Gambar 2: Melihat 5 data teratas dan terbawah

2. **Melihat deskripsi statistik dengan df.describe**

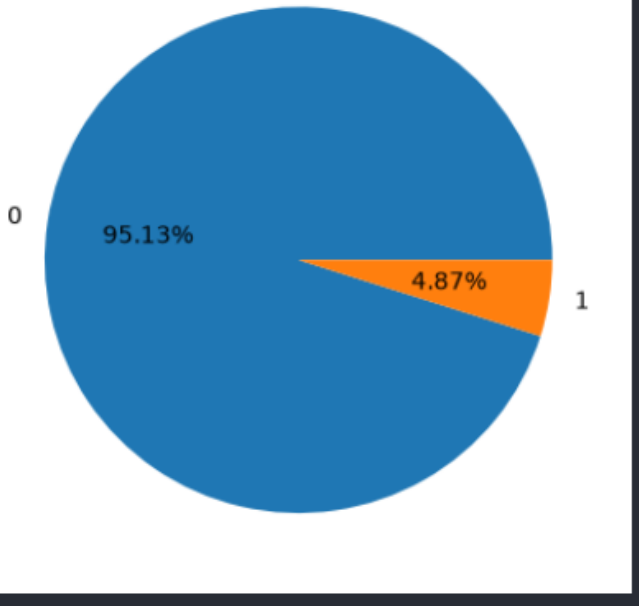
```
df.describe()
```
✓ 0.0s Python

	id	age	hypertension	heart_disease	avg_glucose_level	bmi	stroke
count	5110.000000	5110.000000	5110.000000	5110.000000	5110.000000	4909.000000	5110.000000
mean	36517.829354	43.226614	0.097456	0.054012	106.147677	28.893237	0.048728
std	21161.721625	22.612647	0.296607	0.226063	45.283560	7.854067	0.215320
min	67.000000	0.080000	0.000000	0.000000	55.120000	10.300000	0.000000
25%	17741.250000	25.000000	0.000000	0.000000	77.245000	23.500000	0.000000
50%	36932.000000	45.000000	0.000000	0.000000	91.885000	28.100000	0.000000
75%	54682.000000	61.000000	0.000000	0.000000	114.090000	33.100000	0.000000
max	72940.000000	82.000000	1.000000	1.000000	271.740000	97.600000	1.000000

Gambar 3: Deskripsi Statistik

3. **Melihat presentase target kelas**

```
data = df['stroke'].value_counts()
data.plot(kind="pie", autopct='%.2f%%')
plt.show()
```
✓ 0.2s

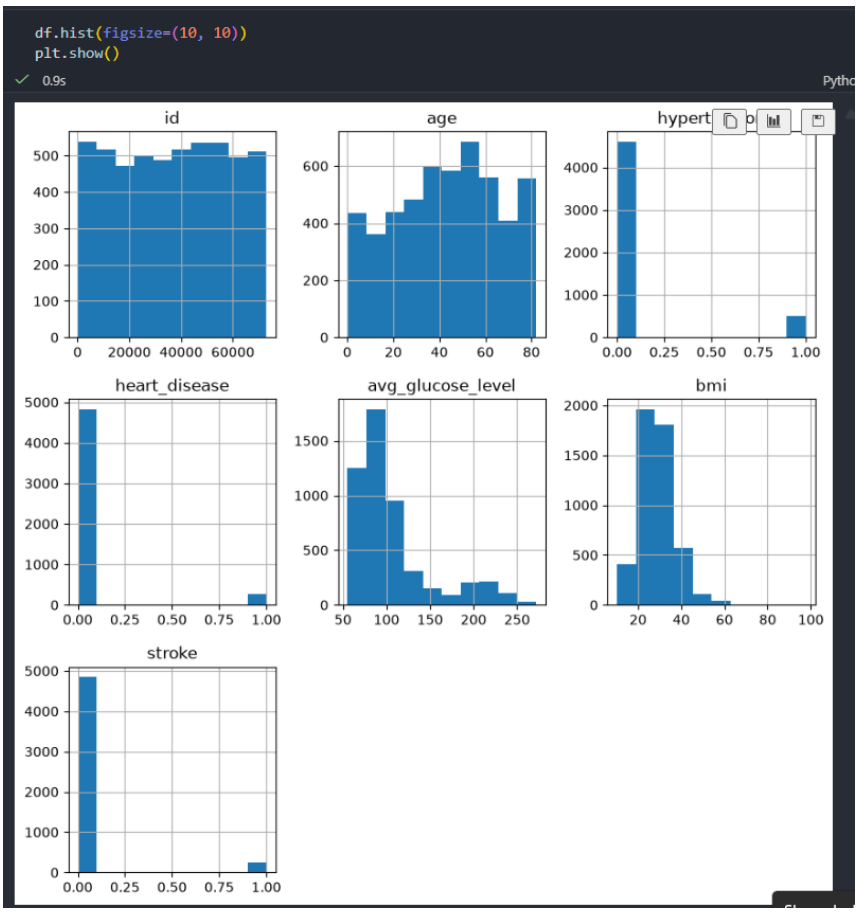


Gambar 4: Presentase Target Kelas

Berdasarkan gambar 4, Kode tersebut digunakan untuk menghitung frekuensi

setiap kelas pada atribut *stroke* menggunakan fungsi `value_counts()`, kemudian memvisualisasikannya dalam bentuk *pie chart*. Parameter `autopct='%.2f%%'` digunakan untuk menampilkan persentase dengan dua angka di belakang koma, sedangkan `plt.show()` digunakan untuk menampilkan grafik. Berdasarkan hasil visualisasi, kelas *stroke* = 0 memiliki proporsi sebesar 95.13%, sedangkan kelas *stroke* = 1 sebesar 4.87%. Dengan demikian, dataset memiliki **ketidakseimbangan kelas (*class imbalance*)**, karena jumlah data pada kelas *stroke* = 0 jauh lebih besar dibandingkan kelas *stroke* = 1.

4. **Melihat Histogram**

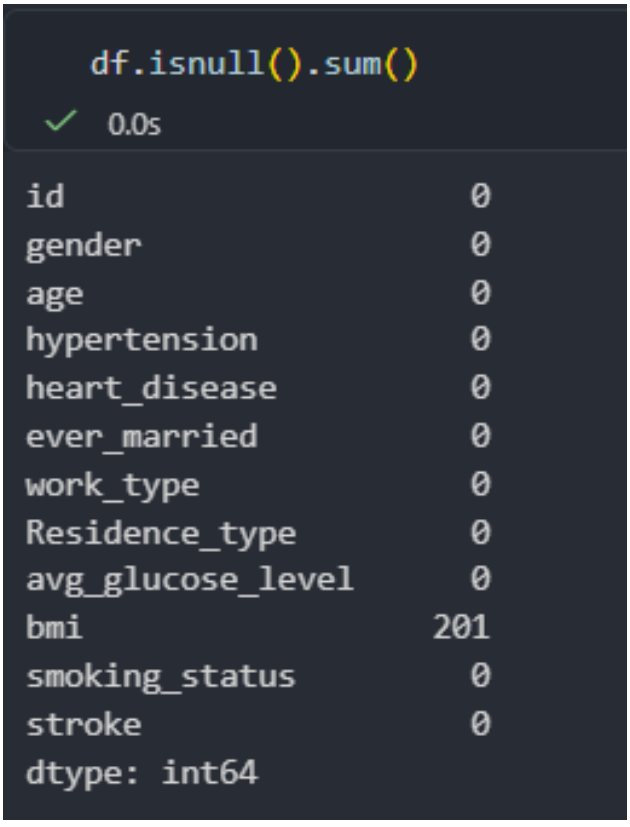


Gambar 5: Histogram

Berdasarkan gambar 5, Kode `df.hist(figsize=(10,10))` digunakan untuk

membuat histogram dari seluruh atribut numerik pada *DataFrame df*, sedangkan `figsize=(10,10)` mengatur ukuran visualisasi. Berdasarkan histogram, atribut `age`, `avg_glucose_level`, dan `bmi` memiliki distribusi yang bervariasi, sedangkan atribut `hypertension`, `heart_disease`, dan `stroke` didominasi oleh nilai 0. Histogram `id` menunjukkan bahwa data tersebar pada rentang nilai ID. Visualisasi ini digunakan untuk memahami distribusi dan karakteristik setiap atribut sebelum dilakukan proses analisis atau pemodelan.

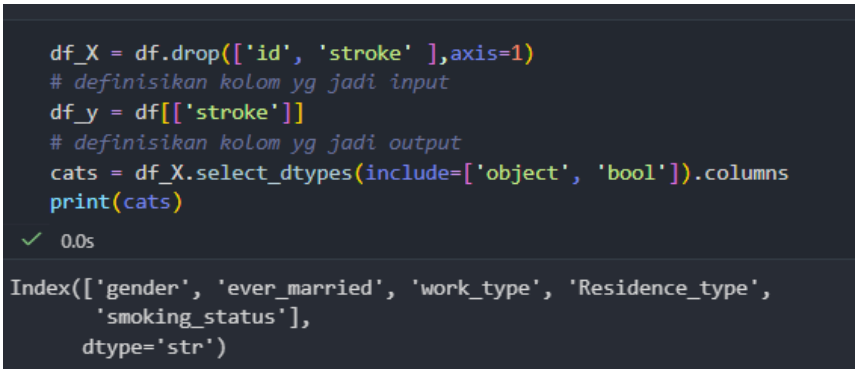
5. **Mengecek missing value dengan df.isnull().sum()**



Gambar 6: Mengecek Missing Value

Berdasarkan pengecekan tersebut, terdapat missing value pada kolom `bmi` sebanyak 201 data.

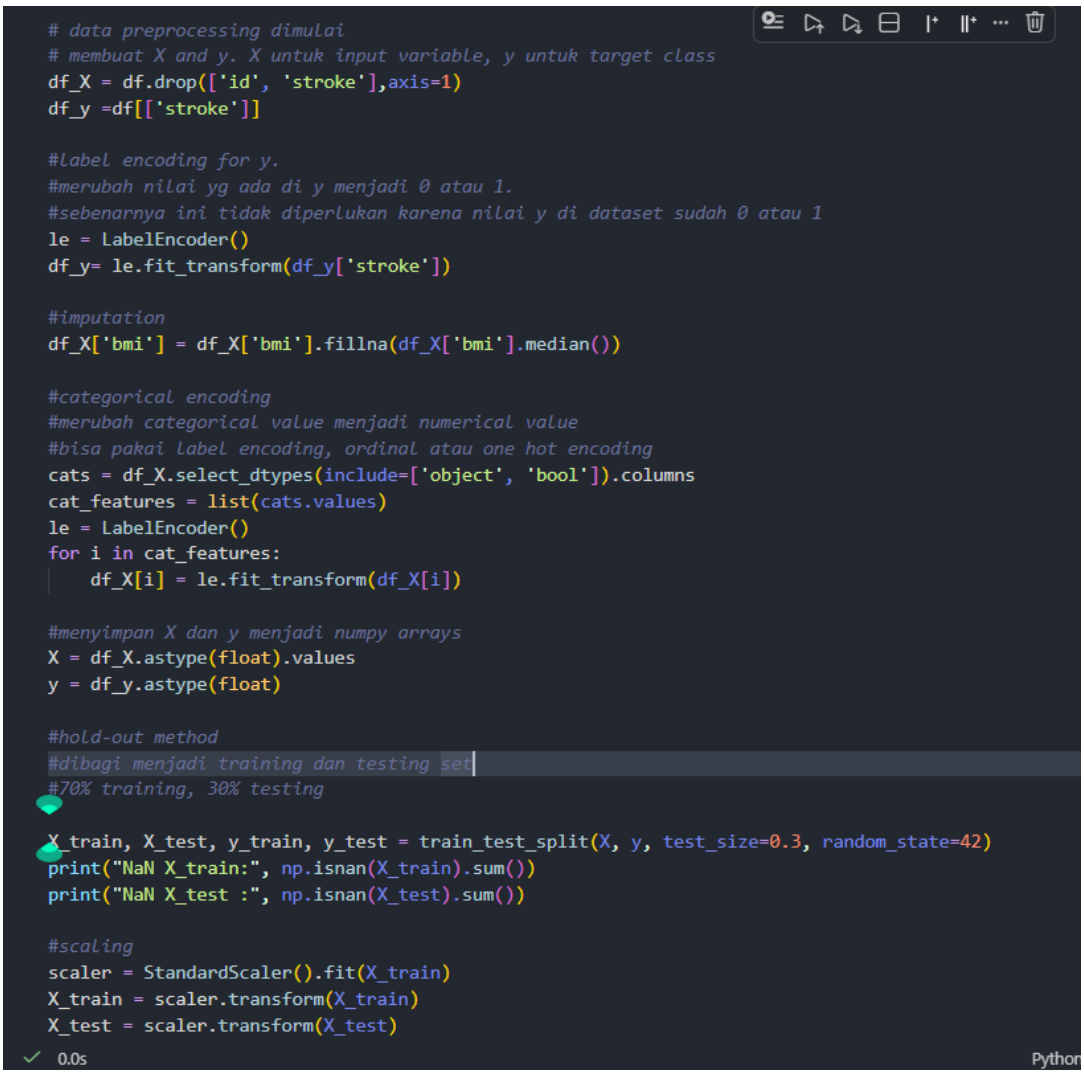
6. **Mengecek atribut kategorikal**



Gambar 7: Mengecek Atribut Kategorikal

Berdasarkan gambar 7, atribut kategorikal yang terdapat pada dataset ini adalah gender, ever_married, work_type, Residence_type, dan smoking_status.

C.1.4 Data Preprocessing



```
# data preprocessing dimulai
# membuat X and y. X untuk input variable, y untuk target class
df_X = df.drop(['id', 'stroke'],axis=1)
df_y =df[['stroke']]

#label encoding for y.
#merubah nilai yg ada di y menjadi 0 atau 1.
#sebenarnya ini tidak diperlukan karena nilai y di dataset sudah 0 atau 1
le = LabelEncoder()
df_y= le.fit_transform(df_y['stroke'])

#imputation
df_X['bmi'] = df_X['bmi'].fillna(df_X['bmi'].median())

#categorical encoding
#merubah categorical value menjadi numerical value
#bisa pakai label encoding, ordinal atau one hot encoding
cats = df_X.select_dtypes(include=['object', 'bool']).columns
cat_features = list(cats.values)
le = LabelEncoder()
for i in cat_features:
    df_X[i] = le.fit_transform(df_X[i])

#menyimpan X dan y menjadi numpy arrays
X = df_X.astype(float).values
y = df_y.astype(float)

#hold-out method
#dibagi menjadi training dan testing set
#70% training, 30% testing

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
print("NaN X_train:", np.isnan(X_train).sum())
print("NaN X_test :", np.isnan(X_test).sum())

#scaling
scaler = StandardScaler().fit(X_train)
X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```


Gambar 8: Data Preprocessing

1. **Feature and Target Separation** Memisahkan data menjadi variabel input X dan variabel target y. Atribut `id` dan `stroke` dihapus dari X, sedangkan `stroke` digunakan sebagai target klasifikasi.

2. **Label Encoding** Mengubah nilai pada variabel target y menjadi bentuk numerik 0 dan 1. Pada dataset ini, proses tersebut sebenarnya tidak diperlukan karena nilai `stroke` sudah berupa 0 dan 1.

3. **Imputation** Mengatasi *missing value* pada atribut `bmi` dengan menggantinya

menggunakan nilai median. Median dipilih agar lebih tahan terhadap pengaruh *outlier*.

4. **Categorical Encoding** Mengidentifikasi atribut kategorikal bertipe `object` atau `bool`, kemudian mengubah nilainya menjadi representasi numerik menggunakan *Label Encoding* agar dapat digunakan oleh algoritma *machine learning*.

5. **Data Type Conversion** Mengubah seluruh nilai X dan y menjadi tipe data numerik `float` dan menyimpannya dalam bentuk *NumPy array*.

6. **Hold-Out Method** Membagi dataset menjadi data *training* sebesar 70% dan data *testing* sebesar 30%. Parameter `random_state=42` digunakan agar pembagian data dapat direproduksi.

7. **Missing Value Checking** Memeriksa jumlah nilai `NaN` pada X_{train} dan X_{test} untuk memastikan tidak terdapat *missing value* setelah proses imputasi.

8. **Feature Scaling** Melakukan standardisasi terhadap fitur menggunakan *StandardScaler* dengan rumus:

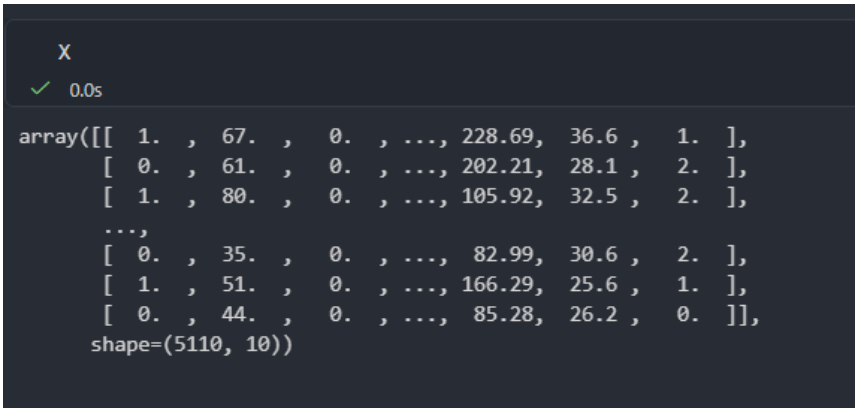
$$z = \frac{x - \mu}{\sigma}$$

sehingga fitur memiliki rata-rata 0 dan standar deviasi 1. Parameter skala dihitung dari data *training* dan kemudian diterapkan pada data *testing* untuk mencegah *data leakage*.

Tahapan preprocessing yang telah dilakukan memastikan bahwa dataset berada dalam kondisi optimal sebelum digunakan untuk membangun model klasifikasi. Langkah berikutnya, dilakukan proses pemeriksaan hasil preprocessing untuk memastikan seluruh transformasi sudah sesuai.

1. **Variabel input X**

   Seluruh nilai kategorik berhasil diubah menjadi representasi numerik, dan tidak terdapat nilai `NaN`.



Gambar 9: Variabel Input X

2. **Variabel target y**



Gambar 10: Variabel Target y

Variabel target y merupakan variabel yang ingin diprediksi oleh model *machine learning*, yaitu atribut `stroke`. Berdasarkan hasil tersebut, y memiliki sebanyak 5110 data dengan dua kelas, yaitu 0 dan 1. Nilai 0 menunjukkan bahwa individu **tidak mengalami stroke**, sedangkan nilai 1 menunjukkan bahwa individu **mengalami stroke**. Dengan demikian, masalah yang digunakan merupakan **klasifikasi biner** karena hanya terdapat dua kelas target.

3. **Data training yang telah dinormalisasi (X_train)**

```
X_train
✓ 0.0s
array([[ 1.18418048, -1.7467638 , -0.31719928, ..., -0.340693  ,
        -1.64591683, -1.29622579],
       [ 1.18418048, -0.63635252, -0.31719928, ...,  2.26654137,
        -0.78843307,  1.52066342],
       [ 1.18418048,  0.02989425,  3.15259225, ..., -0.32155489,
        -0.30772248,  0.58170035],
       ...,
       [-0.84446587, -1.87290652, -0.31719928, ..., -0.18803315,
        -1.43804198, -1.29622579],
       [ 1.18418048,  1.62888649, -0.31719928, ...,  2.01062472,
         0.27692553, -0.35726272],
       [-0.84446587,  0.11872715, -0.31719928, ..., -0.12416526,
         2.78441591,  1.52066342]], shape=(3577, 10))
```

Gambar 11: Data Training yang Telah Dinormalisasi (X_train)

Variabel X_{train} merupakan data fitur yang digunakan untuk melatih model. Berdasarkan hasil tersebut, terdapat 3577 data dengan 10 atribut, sehingga memiliki bentuk $(3577, 10)$. Nilai pada fitur telah melalui proses *feature scaling* menggunakan *StandardScaler*, sehingga nilainya telah distandardisasi dengan rata-rata mendekati 0 dan standar deviasi mendekati 1.

4. **Data testing yang telah dinormalisasi (X_test)**

```
X_test
✓ 0.0s
array([[ 1.18418048, -0.54751962, -0.31719928, ..., -0.90971812,
        -0.76244872, -1.29622579],
       [ 1.18418048, -0.14777156, -0.31719928, ..., -0.89992653,
        -0.07386327,  0.58170035],
       [-0.84446587, -1.569098  , -0.31719928, ..., -0.69675096,
        -0.82740961, -1.29622579],
       ...,
       [ 1.18418048, -0.05893866, -0.31719928, ..., -0.26569829,
        -0.21677723,  0.58170035],
       [-0.84446587,  0.60730811, -0.31719928, ..., -0.80846414,
        -0.63252693, -1.29622579],
       [-0.84446587,  0.74055746, -0.31719928, ..., -0.72746095,
        -0.46362862,  0.58170035]], shape=(1533, 10))
```

Gambar 12: Data Testing yang Telah Dinormalisasi (X_test)

Variabel X_{test} merupakan data fitur yang digunakan untuk menguji kinerja model setelah proses pelatihan. Berdasarkan hasil tersebut, terdapat 1533 data dengan 10 atribut, sehingga memiliki bentuk $(1533, 10)$. Data ini juga telah melalui proses *feature scaling* menggunakan parameter yang diperoleh dari X_{train}.

C.1.5 Model Logistic Regression

```
model=LogisticRegression()
model.fit(X_train, y_train)

# membuat prediksi
y_pred = model.predict(X_test)

# y_pred jika di print out keluar vector prediksi nya
#menghitung performa model, dengan accuracy dll
print('Accuracy ',accuracy_score(y_test, y_pred))
print('Precision ',precision_score(y_test, y_pred, average='macro'))
print('Recall ', recall_score(y_test, y_pred, average='macro'))
print('Confusion matrix ', confusion_matrix(y_test, y_pred))

# plot_confusion_matrix(model, X_test, y_test, cmap=plt.cm.Blues)
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))

# plt.show()
disp.plot(cmap=plt.cm.Blues)

print('F1 ',f1_score(y_test, y_pred, average='macro'))
```

Gambar 13: Model Logistic Regression

1. **Model Training** Model *Logistic Regression* dilatih menggunakan X_{train} dan y_{train} untuk mempelajari hubungan antara fitur dan target.

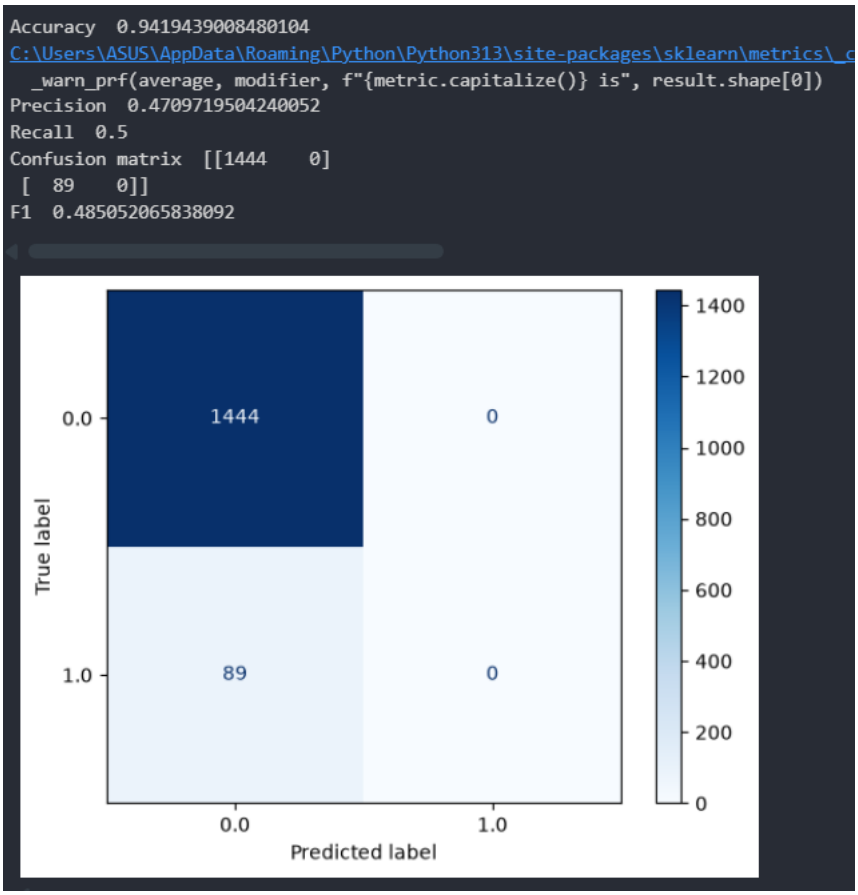
2. **Prediction** Model digunakan untuk memprediksi data pengujian sehingga menghasilkan y_{pred}, yaitu hasil prediksi kelas `stroke` pada X_{test}.

3. **Model Evaluation** Kinerja model dievaluasi menggunakan *Accuracy*, *Precision*, *Recall*, dan *F1-Score*. Selain itu, *Confusion Matrix* digunakan untuk melihat jumlah prediksi benar dan salah pada setiap kelas.

4. **Confusion Matrix Visualization** *Confusion Matrix* divisualisasikan menggunakan `ConfusionMatrixDisplay` untuk mempermudah interpretasi hasil

klasifikasi.

5. **F1-Score** *F1-Score* digunakan untuk mengukur keseimbangan antara *Precision* dan *Recall*. Pada kode ini digunakan `average='macro'`, sehingga skor dihitung dengan memberikan bobot yang sama pada setiap kelas.



Gambar 14: Confusion Matrix Model Logistic Regression

C.1.6 Model K-Nearest Neighbors (KNN)

```python
model=KNeighborsClassifier(n_neighbors=10)
model.fit(X_train, y_train)

# membuat prediksi
y_pred = model.predict(X_test)

#menghitung performa model, dengan accuracy dll
print('Accuracy ',accuracy_score(y_test, y_pred))
print('Precision ',precision_score(y_test, y_pred, average='macro'))
print('Recall ',recall_score(y_test, y_pred, average='macro'))
print('Confusion matrix ', confusion_matrix(y_test, y_pred))

# plot_confusion_matrix(model, X_test, y_test, cmap=plt.cm.Blues)
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))

# plt.show()
disp.plot(cmap=plt.cm.Blues)
```

Gambar 15: Model K-Nearest Neighbors (KNN)

1. **Model Training** Model *K-Nearest Neighbors (KNN)* dibuat dengan $n_neighbors = 10$, sehingga model menggunakan 10 tetangga terdekat dalam menentukan kelas data.

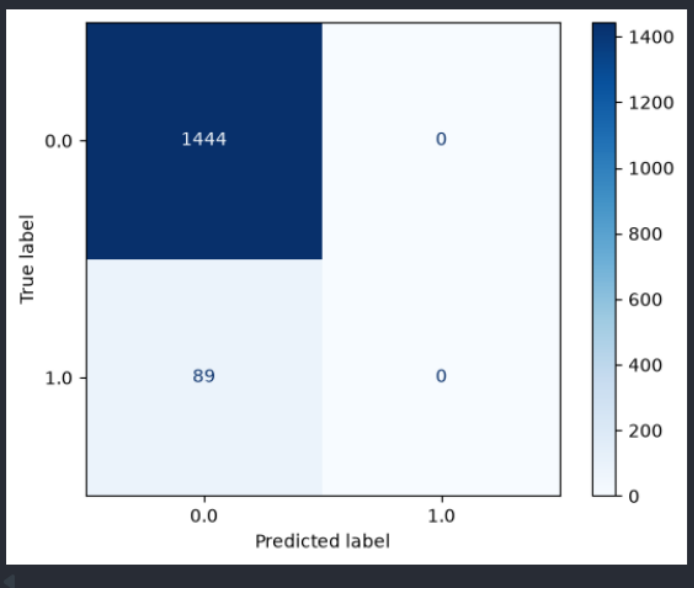
2. **Prediction** Model yang telah dilatih digunakan untuk memprediksi X_{test}, kemudian menghasilkan y_{pred} sebagai hasil prediksi kelas `stroke`.

3. **Model Evaluation** Kinerja model dievaluasi menggunakan *Accuracy*, *Precision*, *Recall*, dan *Confusion Matrix*. Parameter `average='macro'` digunakan agar setiap kelas memperoleh bobot yang sama dalam perhitungan *Precision* dan *Recall*.

4. **Confusion Matrix Visualization** *Confusion Matrix* dihitung berdasarkan y_{test} dan y_{pred}, kemudian divisualisasikan menggunakan `ConfusionMatrixDisplay` untuk melihat hasil klasifikasi pada masing-masing kelas.

```
Accuracy  0.9419439008480104
Precision  0.4709719504240052
Recall  0.5
Confusion matrix  [[1444    0]
 [  89    0]]
C:\Users\ASUS\AppData\Roaming\Python\Python313\site-packages\sklearn\metrics\_classif
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1cba3e13110>
```



Gambar 16: Confusion Matrix Model K-Nearest Neighbors (KNN)

C.1.7 Model Decision Tree

```python
model=DecisionTreeClassifier(criterion="entropy")
model. fit(X_train, y_train)

# membuat prediksi
y_pred = model.predict(X_test)

#menghitung performa model, dengan accuracy dll
print('Accuracy ',accuracy_score(y_test, y_pred))
print('Precision ',precision_score(y_test, y_pred, average='macro'))
print('Recall ', recall_score(y_test, y_pred, average='macro'))
print('Confusion matrix ', confusion_matrix(y_test, y_pred))

# plot_confusion_matrix(model, X_test, y_test, cmap=plt.cm.Blues)
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))

# plt.show()
disp.plot(cmap=plt.cm.Blues)
```

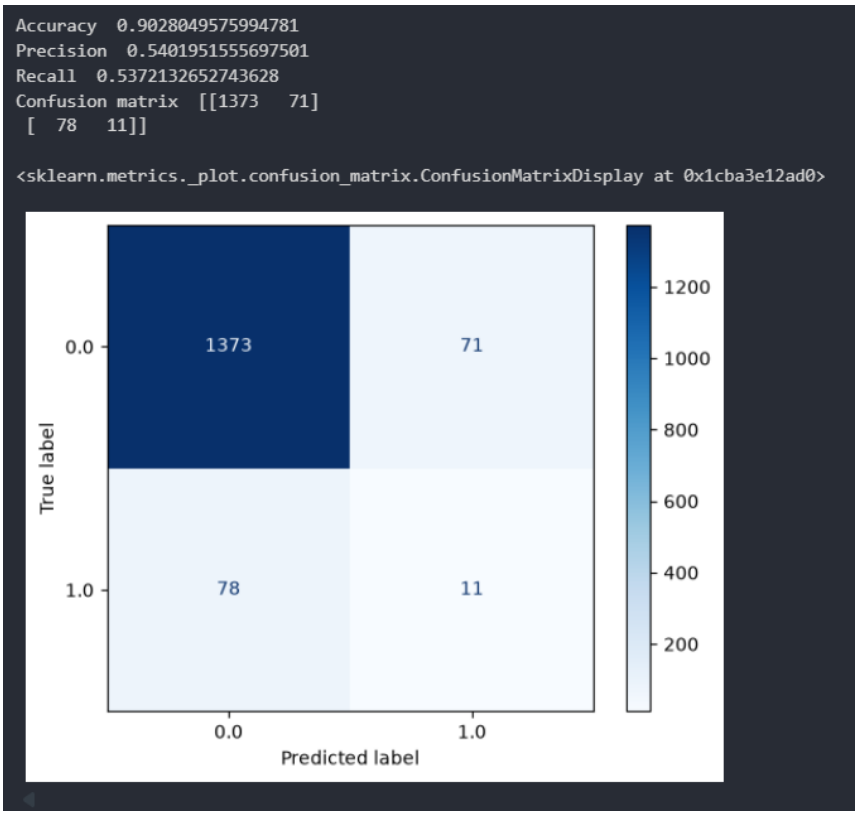
Gambar 17: Model Decision Tree

1. **Model Training** Model *Decision Tree* dibuat menggunakan `criterion="entropy"`, sehingga pemisahan data pada setiap node ditentukan berdasarkan nilai *Entropy* dan *Information Gain*.

2. **Prediction** Model yang telah dilatih digunakan untuk memprediksi X_{test}, sehingga menghasilkan y_{pred} sebagai hasil prediksi kelas `stroke`.

3. **Model Evaluation** Kinerja model dievaluasi menggunakan *Accuracy*, *Precision*, *Recall*, dan *Confusion Matrix*. Parameter `average='macro'` memberikan bobot yang sama pada setiap kelas dalam perhitungan *Precision* dan *Recall*.

4. **Confusion Matrix Visualization** *Confusion Matrix* dihitung berdasarkan y_{test} dan y_{pred}, kemudian divisualisasikan menggunakan `ConfusionMatrixDisplay` untuk melihat jumlah prediksi yang benar dan salah pada masing-masing kelas.



Gambar 18: Confusion Matrix Model Decision Tree

C.1.8 Model Random Forest

```
model=RandomForestClassifier()
model.fit(X_train, y_train)

# membuat prediksi
y_pred = model.predict(X_test)

#menghitung performa model, dengan accuracy dll
print('Accuracy ',accuracy_score(y_test, y_pred))
print('Precision ',precision_score(y_test, y_pred, average='macro'))
print('Recall ', recall_score(y_test, y_pred, average='macro'))
print('Confusion matrix ', confusion_matrix(y_test, y_pred))

# plot_confusion_matrix(model, X_test, y_test, cmap=plt.cm.Blues)
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))

# plt.show()
disp.plot(cmap=plt.cm.Blues)
```

Gambar 19: Model Random Forest

1. **Model Training** Model *Random Forest* dibuat dan dilatih menggunakan X_{train} dan y_{train}. Model ini merupakan kumpulan beberapa *Decision Tree* yang bekerja secara bersama-sama untuk menghasilkan prediksi.

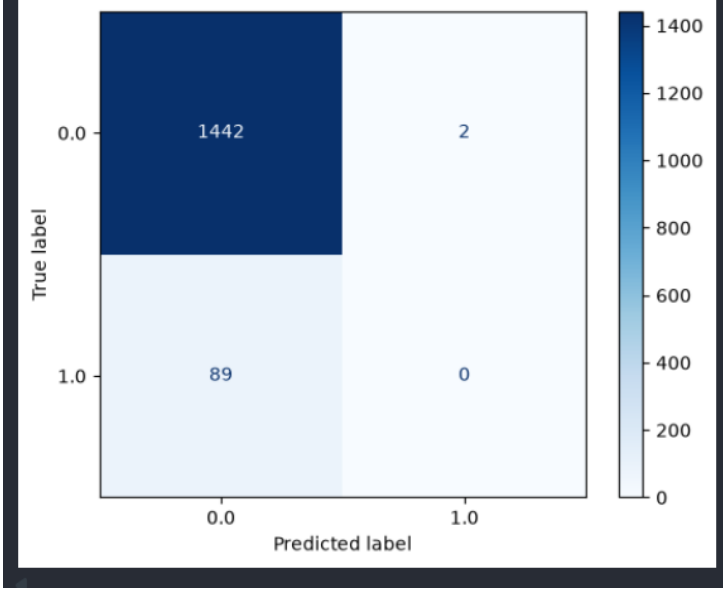
2. **Prediction** Model yang telah dilatih digunakan untuk memprediksi X_{test}, sehingga menghasilkan y_{pred} sebagai hasil prediksi kelas `stroke`.

3. **Model Evaluation** Kinerja model dievaluasi menggunakan *Accuracy*, *Precision*, *Recall*, dan *Confusion Matrix*. Parameter `average='macro'` memberikan bobot yang sama pada setiap kelas dalam perhitungan *Precision* dan *Recall*.

4. **Confusion Matrix Visualization** *Confusion Matrix* dihitung berdasarkan y_{test} dan y_{pred}, kemudian divisualisasikan menggunakan `ConfusionMatrixDisplay` untuk melihat jumlah prediksi benar dan salah pada masing-masing kelas.

```
Accuracy  0.9406392694063926
Precision  0.47093403004572176
Recall  0.49930747922437674
Confusion matrix  [[1442    2]
 [  89    0]]

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x1cba4200d60>
```



Gambar 20: Confusion Matrix Model Random Forest

C.1.9 Ada Boost

```python
model=AdaBoostClassifier()
model.fit(X_train, y_train)

# membuat prediksi
y_pred = model.predict(X_test)

#menghitung performa model, dengan accuracy dll
print('Accuracy ',accuracy_score(y_test, y_pred))
print('Precision ',precision_score(y_test, y_pred, average='macro'))
print('Recall ', recall_score(y_test, y_pred, average='macro'))
print('Confusion matrix ', confusion_matrix(y_test, y_pred))

# plot_confusion_matrix(model, X_test, y_test, cmap=plt.cm.Blues)
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=np.unique(y_test))

# plt.show()
disp.plot(cmap=plt.cm.Blues)
```

Gambar 21: Model Ada Boost

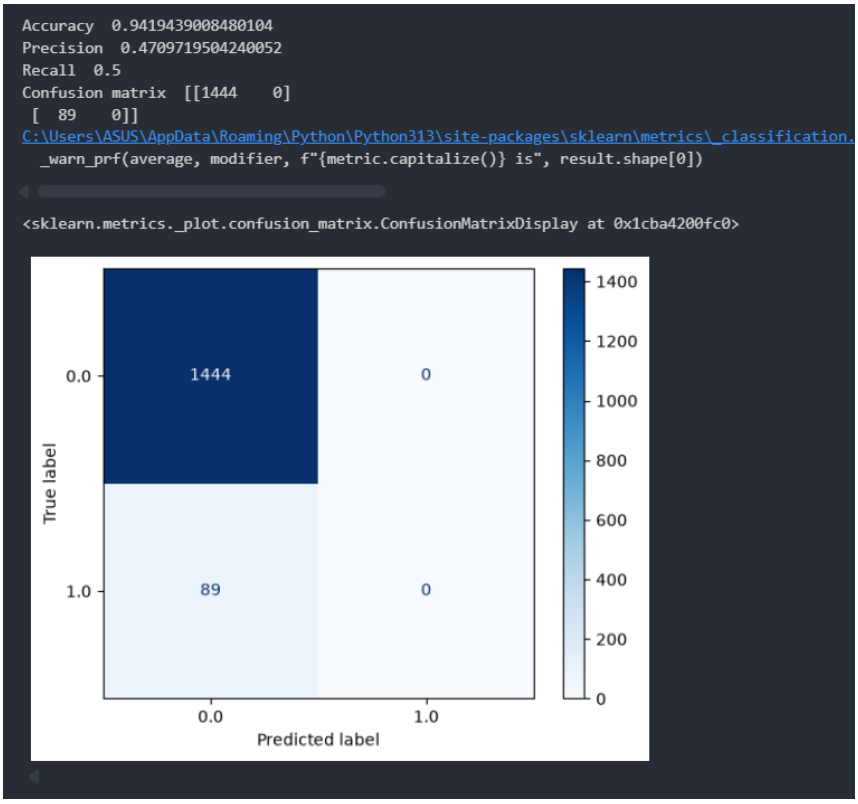
1. **Model Training** Model *AdaBoost* dibuat dan dilatih menggunakan X_{train} dan y_{train}. AdaBoost menggabungkan beberapa *weak learner* secara bertahap

untuk meningkatkan kemampuan klasifikasi.

2. **Prediction** Model digunakan untuk memprediksi X_{test}, sehingga menghasilkan y_{pred} sebagai hasil prediksi kelas `stroke`.

3. **Model Evaluation** Kinerja model dievaluasi menggunakan *Accuracy*, *Precision*, *Recall*, dan *Confusion Matrix*. Parameter `average='macro'` memberikan bobot yang sama pada setiap kelas.

4. **Confusion Matrix Visualization** *Confusion Matrix* dihitung berdasarkan y_{test} dan y_{pred}, kemudian divisualisasikan menggunakan `ConfusionMatrixDisplay` untuk melihat jumlah prediksi benar dan salah pada setiap kelas.



Gambar 22: Confusion Matrix Model Ada Boost

C.2 Tugas dan Analisis

Diberikan dataset dengan link berikut `https://www.kaggle.com/datasets/bhavikjikadara/loan-status-prediction`

C.2.1 Tujuan Penggunaan Dataset

Dataset Loan Status Prediction digunakan untuk membangun sistem otomatisasi evaluasi kelayakan pinjaman nasabah pada institusi perbankan atau finansial. Tujuannya adalah memprediksi apakah pengajuan kredit seorang pemohon layak disetujui atau ditolak berdasarkan profil demografis, kapasitas finansial, dan riwayat kredit mereka, sehingga bank dapat menekan risiko kredit macet dan mempercepat proses persetujuan kredit.

C.2.2 Atribut Input dan Output

Variabel input X merupakan atribut yang digunakan sebagai dasar prediksi, yaitu `Gender`, `Married`, `Dependents`, `Education`, `Self_Employed`, `ApplicantIncome`, `CoapplicantIncome`, `LoanAmount`, `Loan_Amount_Term`, `Credit_History`, dan `Property_Area`. Atribut `Loan_ID` tidak digunakan sebagai input karena hanya merupakan identitas data. Variabel output y adalah `Loan_Status`, dengan Y menunjukkan pinjaman disetujui dan N menunjukkan pinjaman ditolak.

C.2.3 Model Klasifikasi

```
import pandas as pd
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score,
    precision_score, recall_score
```

```
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder,
    StandardScaler
from sklearn.tree import DecisionTreeClassifier

# Memuat dataset
df = pd.read_csv('loan_dataset/loan_data.csv')

# Menghapus kolom Loan_ID jika ada
if 'Loan_ID' in df.columns:
  df = df.drop(columns=['Loan_ID'])

# Pemisahan fitur dan target
X = df.drop(columns=['Loan_Status'])
y = df['Loan_Status'].map({'Y': 1, 'N': 0})

# Pembagian data latih dan data uji
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Definisi kolom numerik dan kategorikal
num_cols = [
    'ApplicantIncome',
    'CoapplicantIncome',
    'LoanAmount',
    'Loan_Amount_Term',
]
```

```
cat_cols = [
    'Gender',
    'Married',
    'Dependents',
    'Education',
    'Self_Employed',
    'Credit_History',
    'Property_Area',
]

# Pipeline preprocessing
num_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler()),
])

cat_pipeline = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OneHotEncoder(handle_unknown='ignore')),
])

preprocessor = ColumnTransformer([
    ('num', num_pipeline, num_cols),
    ('cat', cat_pipeline, cat_cols),
])

# Daftar model klasifikasi
models = {
    'Logistic Regression':
        LogisticRegression(random_state=42),
```

```
    'KNN': KNeighborsClassifier(n_neighbors=5),
    'Decision Tree':
        DecisionTreeClassifier(random_state=42,
        max_depth=5),
    'Random Forest':
        RandomForestClassifier(n_estimators=100,
        random_state=42),
}

# Pelatihan dan evaluasi
results = []
for name, model in models.items():
  pipeline = Pipeline([('prep', preprocessor), ('clf',
     model)])
  pipeline.fit(X_train, y_train)
  y_pred = pipeline.predict(X_test)

  results.append({
      'Model': name,
      'Accuracy': accuracy_score(y_test, y_pred),
      'Precision': precision_score(y_test, y_pred),
      'Recall': recall_score(y_test, y_pred),
  })
```

Outputnya sebagai berikut

	Model	Accuracy	Precision	Recall
0	Logistic Regression	0.8943	0.9238	0.9510
1	KNN	0.8780	0.8850	0.9804
2	Decision Tree	0.8780	0.9143	0.9412
3	Random Forest	0.8862	0.9231	0.9412

Gambar 23: Output Evaluasi Model Klasifikasi

C.2.4 Perbandingan performa model

Table 1: Performa Model Klasifikasi Prediksi Kelayakan Pinjaman

Model	**Accuracy**	**Precision**	**Recall**
Logistic Regression	0.8943	0.9238	0.9510
Random Forest	0.8862	0.9231	0.9412
KNN	0.8780	0.8850	0.9804
Decision Tree	0.8780	0.9143	0.9412

C.2.5 Model Paling Baik

1. **Akurasi dan Keseimbangan Metrik Tertinggi** *Logistic Regression* mencapai akurasi tertinggi sebesar 89.43%, serta mempertahankan nilai *precision* dan *recall* di atas 92%.

2. **Sifat Fitur Dominan** Pada dataset persetujuan kredit, fitur `Credit_History` memiliki peranan penting dalam menentukan probabilitas persetujuan. Algoritma linear seperti *Logistic Regression* mampu memanfaatkan hubungan tersebut secara optimal.

3. **Minim Risiko Overfitting** Dataset memiliki jumlah observasi yang relatif terbatas, yaitu sekitar 600 data. *Logistic Regression* memiliki kompleksitas model yang relatif rendah sehingga cenderung memiliki risiko *overfitting* yang lebih kecil dibandingkan model pohon yang tidak dibatasi kedalamannya.

4. **Transparansi Regulasi** Dalam industri perbankan, *Logistic Regression* lebih mudah diinterpretasikan melalui nilai koefisien setiap fitur. Hal ini dapat membantu proses audit dan memberikan penjelasan mengenai faktor yang memengaruhi keputusan persetujuan kredit.

Berdasarkan hasil eksperimen tersebut, ***Logistic Regression*** **dipilih sebagai model terbaik** karena memberikan performa yang paling baik sekaligus memiliki interpretabilitas dan risiko *overfitting* yang relatif rendah.

D Kesimpulan

Berdasarkan hasil praktikum klasifikasi, dapat disimpulkan bahwa proses klasifikasi dapat digunakan untuk membangun model yang mampu memprediksi kelas berdasarkan karakteristik data yang tersedia. Tahapan *data preprocessing*, seperti *imputation*, *categorical encoding*, pembagian data menjadi data latih dan data uji, serta *feature scaling*, merupakan tahap penting untuk memastikan data dapat diproses dengan baik oleh algoritma klasifikasi. Beberapa algoritma yang digunakan, yaitu *Logistic Regression*, KNN, *Decision Tree*, dan *Random Forest*, menghasilkan performa yang berbeda berdasarkan metrik *accuracy*, *precision*, dan *recall*.

Dari hasil evaluasi, *Logistic Regression* memberikan performa terbaik dengan *accuracy* sebesar 89.43%, *precision* sebesar 92.38%, dan *recall* sebesar 95.10%. Hasil tersebut menunjukkan bahwa *Logistic Regression* mampu melakukan klasifikasi dengan baik dibandingkan model lainnya pada dataset yang digunakan. Oleh karena itu, *Logistic Regression* dapat dipilih sebagai model yang paling sesuai dalam praktikum ini, dengan mempertimbangkan performa klasifikasi yang tinggi serta kompleksitas model yang relatif sederhana dan mudah diinterpretasikan.

E Daftar Pustaka

[1] K. N. Fadhlurrohman, "Pengertian klasifikasi dalam data mining," https://bis-sby.telkomuniversity.ac.id/pengertian-klasifikasi-dalam-data-mining/, 2026, [Accessed 04-09-2026].

[2] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Waltham, MA: Morgan Kaufmann, 2011, dasar teori mengenai klasifikasi dan penggalian data.

[3] T. Fawcett, "An introduction to roc analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006, dasar teori untuk matriks evaluasi, metrik presisi, recall, dan confusion matrix.